

Accelerate QC-LDPC FEC for High-Throughput 5G/6G Wireless & Satellite on CPU and GPU SDRs

Vasiliy S. Usatyuk
Department of Computer Science
South-West State University
Kursk, Russia
Email: L@Lcrypto.com

Sergey I. Egorov
Department of Computer Science
South-West State University
Kursk, Russia
Email: sie58@mail.ru

Abstract—This paper presents a high-throughput decoding method for quasi-cyclic low-density parity-check (QC-LDPC) codes, optimized for both CPU and GPU-based Software-Defined Radios (SDRs). The approach leverages a SIMD data alignment architecture that minimizes cache misses and is designed to take full advantage of modern CPUs and GPUs with rich registers and caches. An innovative decoding algorithm is introduced, exploiting the SIMD capabilities of contemporary CPUs to enhance parallelism, while optimizing memory access patterns to align SIMD operations and minimize branching and cache misses on AMD EPYC CPUs (2xEPYC-9754). This results in significant throughput improvements. The method is further extended to GPUs, utilizing their large-scale parallelism through efficient use of registers and local memory. To maximize performance, a memory-efficient check node decoding strategy is employed, ensuring that operations fit within the cache of the Streaming Multiprocessors (SM) to reduce memory access overhead. Simulation results demonstrate the effectiveness of this approach: the CPU-based 8-iteration layered min-sum decoder with early termination achieves coded bit throughputs of up to 72.9 Gbps for 16200-bit codes and 50.9 Gbps for 64800-bit DVB-S2 QC-LDPC codes. On GPUs, the decoder reaches 132 Gbps for short 802.11n Wi-Fi 1944-bit codes and 22.3 Gbps for 5G Base Graph 1 26112-bit codes on an NVIDIA RTX 4090. For 6G prototype QC-LDPC codes, the proposed 3-layer design achieves up to 76 Gbps for 1920-bit codes, with surpassing 104 Gbps on the NVIDIA RTX 5090. Compared to the open-source AFF3CT CPU framework, the proposed method offers 5 to 12 times higher throughput across various code lengths. Even when AFF3CT is optimized with MIPP intrinsics, the new decoder provides 3.2 to 5.7 times better throughput. The GPU implementation delivers up to 120 times the speed of previously published results and offers 13 to 40 times improvement over older GPUs like the GTX 1080 Ti and Titan X, scaling effectively with modern GPUs like the RTX 4090. This approach establishes new performance benchmarks for QC-LDPC decoding, positioning it as an ideal solution for next-generation 5G/6G wireless and satellite communications.

Keywords—5G NR; 6G prototype; 802.11n Wi-Fi; AFF3CT; AVX; CUDA; DVB-S2; GPU acceleration; QC-LDPC codes; SIMD; high-throughput decoding; memory-efficient algorithms.

I. INTRODUCTION

Effective utilization of register and cache structures, combined with memory alignment and regular processing, enables near-peak performance on SIMD devices such as multi-core CPUs and GPUs. However, the practical implementation of low-density parity-check (LDPC) decoding has been limited by the irregular nature of operations, the constrained register

and cache capacity of multi-core processors and GPUs, and the substantial memory demands for soft metrics. The advent of 5G/6G wireless and satellite communications, particularly from 3GPP Release 18 onward, necessitates high-throughput decoders to meet growing bandwidth requirements. This paper introduces an optimized scheduler for QC-LDPC decoding that achieves unprecedented throughput levels on modern GPUs. For short codes with bit lengths in the thousands, the proposed decoder exceeds 100 Gbps at block error rates near 10^{-5} , satisfying the ultra-reliable low-latency communication (URLLC) standards crucial for 5G and paving the way for future 6G systems, Table I, [1]–[6]. Looking ahead to 2028–2030, anticipated advancements in GPU architectures and high-bandwidth memory (HBM) are expected to elevate decoder performance to a minimum of 100 Gbps (64QAM, 30 GHz bandwidth), with optimistic projections reaching over 1 Tbps. Likewise, multi-core CPUs are projected to surpass 100 Gbps, allowing QC-LDPC decoders to thrive in software-defined radio (SDR) platforms. This advancement reduces dependency on specialized ASICs or RFSoc AMD FPGAs with hardware decoding cores, potentially shifting protocol implementations toward general-purpose computing systems.

TABLE I: General 6G Requirements

Property	Requirement
Throughput	100 Gbps - 1 Tbps
Bandwidth	100 MHz - 1 THz
Frequency Range	100 GHz - 1 THz
Latency	1 ms or less
Connection Density	1 million devices/km ²
Energy Efficiency	100 times more efficient
Reliability	99.9999%
eURLLC	$FER < 10^{-9}$
Mobility Support	Up to 1000 km/h
AI Integration	Fully integrated AI/ML

While arrays offer immense potential, state-of-the-art soft decoding of QC-LDPC codes still predominantly relies on vector SIMD CPUs and GPUs. Notable prior works highlight this progression. For example, [7] achieved a peak throughput of 1800 Mbps for DVB-S2 codes using a dual AMD EPYC-7351 system (64 cores) with AVX-256 support. Similarly, [8], [9] used a 16-SIMD-width AVX implementation with Aff3ct on a XeonPhi-7230 for IEEE 802.3an LDPC decoding, reach-

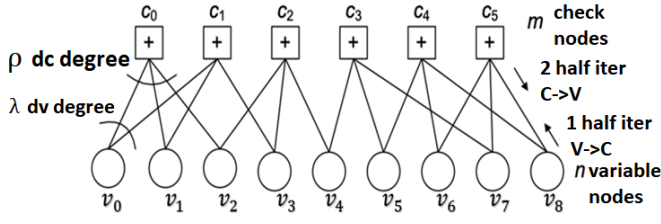


Fig. 1: Message passing under LDPC code

ing 53 Mbps. In [10], a dual Intel XEON-6148 system with 40 cores and AVX-512 support achieved a peak throughput of around 11481 Mbps for DVB-S2 codes. On GPUs, early work such as [16] reported a peak throughput of 913 Mbps for Wi-Fi 802.11n (1944, 972) codes on a GTX TITAN X using flooding Min-Sum decoding with 10 iterations. Throughput increased to 1660 Mbps in a concatenated decoding scheme with BCH codes [17]. Additionally, [18] demonstrated a throughput of 4.7 Gbps for LDPC decoding on a GTX 1080 Ti. While these advancements illustrate the potential of CPUs and GPUs for LDPC decoding, they remain insufficient to meet the stringent demands of next-generation 6G wireless and satellite systems. This work addresses these limitations by introducing novel architectures that fully leverage the potential of LDPC codes, bridging the gap between current capabilities and the throughput requirements of future high-bandwidth communication standards.

II. QC-LDPC CODES AND LAYERED NMS DECODER

A QC-LDPC code is described by a parity-check matrix $\mathbf{H}$ which consists of square blocks which could be either zero matrix or circulant permutation matrices, [24], [25]. Let $P = (P_{ij})$ be the $Z \times Z$ circulant permutation matrix defined by

$$P_{ij} = \begin{cases} 1, & \text{if } i + 1 \equiv j \pmod{Z} \\ 0, & \text{otherwise.} \end{cases}$$

Then P^k is the circulant permutation matrix (CPM) which shifts the identity matrix I to the right by k times for any k , $0 \leq k \leq Z - 1$. For simplicity of notation denote the zero matrix by P^∞ . Denote the set $\{\infty, 0, 1, \dots, Z - 1\}$ by $\mathcal{A}_Z$. Let the matrix $\mathbf{H}$ of size $mZ \times nZ$ be defined in the following manner

$$\mathbf{H} = \begin{bmatrix} P_{a_{11}} & P_{a_{12}} & \cdots & P_{a_{1n}} \\ P_{a_{21}} & P_{a_{22}} & \cdots & P_{a_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ P_{a_{m1}} & P_{a_{m2}} & \cdots & P_{a_{mn}} \end{bmatrix},$$

where $a_{i,j} \in \mathcal{A}_L$. Further we call Z the circulant size of $\mathbf{H}$. In what follows a code C with parity-check matrix $\mathbf{H}$ will be referred to as a QC-LDPC code.

Let $E(\mathbf{H}) = (E_{ij}(\mathbf{H}))$ be the exponent matrix of $\mathbf{H}$ given by:

$$E(\mathbf{H}) = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix},$$

i.e., the entry $E_{ij}(\mathbf{H}) = a_{ij}$. The mother matrix or base matrix, matrix which graph $M(\mathbf{H})$ is a $m \times n$ binary matrix obtained from replacing -1 's and other integers by 0 and 1, respectively, in $E(\mathbf{H})$. The factor graph of the LDPC code, denoted as G and illustrated in Fig. 1, is a Tanner graph comprising two vertex sets. One set encompasses $m = n - k$ factor (check) vertices (nodes) $c_0, c_1, \dots, c_{m-1}$, representing m rows of the matrix $\mathbf{H}$. The second set comprises n variable (code) vertices $\{v_0, v_1, \dots, v_{n-1}\}$, corresponding to the n columns of the matrix $\mathbf{H}$.

Message-passing soft decoding, commonly known as belief propagation (BP) or sum-product decoding, along with its min-sum approximation, is widely used for error correction in LDPC codes [25], [26]. The decoding process involves iterative updates of messages between variable nodes and check nodes in two alternating steps: variable-to-check node updates (V2C) and check-to-variable (C2V) node updates, Fig. 1. For efficient hardware implementation, the decoder algorithm employs a fixed-point quantization approach, which restricts the range of message values to optimize hardware processing. The quantization limits are set as follows: variable nodes (VN_{clipping}) are represented using q -bit integers, with $VN_{\text{clipping}} = 2^{q-1} - 1$, while check nodes (CN_{clipping}) are represented using p -bit integers, with $CN_{\text{clipping}} = 2^{p-1} - 1$. The quantized normalized min-sum (QNMS) decoding algorithm, as outlined in Algorithm 1, includes an early termination criterion to enhance efficiency. To avoid encoder bottlenecks and reduce blocks per decoder, we use deeply modify the [19].

III. SIMD SOFT MESSAGES OPTIMIZED DECODING

The proposed decoding architecture utilizes SIMD AVX-256/512 instructions to optimize the performance and efficiency of LDPC decoding, specifically for Quasi-Min-Sum (QNMS) algorithms. By incorporating SIMD AVX vector instructions, the architecture enhances efficiency by simplifying the C2V message updates, reducing memory overhead, and boosting processing speed. The use of shared memory and registers helps optimize memory usage, minimize global memory accesses, and reduce memory bandwidth demands. With SIMD instructions integrated into the design, the architecture maximizes parallelism and throughput, perfectly aligning with the requirements of Min-Sum decoding.

The high-level processing of the proposed approach is illustrated in Algorithm 2, which details the SIMD processing for LDPC decoding. Additionally, Algorithm 3 outlines the LNMS decoder with early termination, showing the details of operations. The proposed method extends Bertrand Le Gal's scalable, high-throughput LDPC decoding framework by incorporating AVX-256/512, SIMD instructions within

Algorithm 1 Layered Normalized Min-Sum (LNMS) LDPC Decoder with Early Termination

```

1: Input:  $E_n$  (LLRs from channel),  $\alpha$  scale parameter,
    $iter_{\max}$ ,  $VN_{\text{clipping}}$ ,  $CN_{\text{clipping}}$ 
2: Output: Corrected codeword  $\mathbf{c}$  or decoding failure
3: Kernel 1: Initialization
4: for  $m = 0$  to  $|CN| - 1$  do
5:   for  $n = 0$  to  $|VN| - 1$  do
6:      $L_{mn}^{(0)} \leftarrow 0$ ,  $c_n \leftarrow 0$ 
7:   end for
8: end for
9: Kernel 2: Iterative LNMS Decoding
10: for  $iter = 0$  to  $iter_{\max} - 1$  do
11:   Kernel 3: CN Initialization
12:   for  $m = 0$  to  $|CN| - 1$  do
13:      $cn_{\text{sign}} \leftarrow 0$ 
14:      $min_1, min_2 \leftarrow VN_{\text{clipping}}, CN_{\text{clipping}}$ 
15:   end for
16:   Kernel 4: CN Update
17:   for  $n = 0$  to  $|VN| - 1$  do
18:      $msg_{\text{marginal}} \leftarrow E_n - L_{mn}^{(iter-1)}$ 
19:      $cn_{\text{sign}} \leftarrow cn_{\text{sign}} \oplus \text{sgn}(msg_{\text{marginal}})$ 
20:      $min_2 \leftarrow \min(\max(\alpha|msg_{\text{marginal}}|, min_1), min_2)$ 
21:      $min_1 \leftarrow \min(|msg_{\text{marginal}}|, min_1)$ 
22:   end for
23:   Kernel 5: VN Update
24:   for  $n = 0$  to  $|VN| - 1$  do
25:      $\text{sgn}(msg_{\text{marginal}}) \leftarrow cn_{\text{sign}} \oplus \text{sgn}(L_{mn}^{(iter)})$ 
26:      $msg_{\text{val}} \leftarrow \begin{cases} min_2, & \text{if } |L_{nm}^{(iter)}| = min_1 \\ min_1, & \text{otherwise} \end{cases}$ 
27:      $L_{mn}^{(iter)} \leftarrow \text{sgn}(msg_{\text{marginal}}) \cdot msg_{\text{val}}$ 
28:      $E_n \leftarrow L_{nm}^{(iter)} + L_{mn}^{(iter)}$ 
29:   end for
30:   Kernel 6: Early Termination Check
31:    $valid \leftarrow \text{true}$ 
32:   for  $m = 0$  to  $|CN| - 1$  do
33:     if  $cn_{\text{sign}} > 0$  then
34:       if  $iter < iter_{\max} - 2$  then
35:         continue
36:       else
37:         fail decoding:  $c_n = \emptyset$ 
38:         return
39:       end if
40:     end if
41:   end for
42:   if  $valid$  then
43:     break
44:   end if
45: end for
46: Kernel 7: Hard Decision (only  $c_i = 1$  mask processing)
47: for  $n = 0$  to  $|VN| - 1$  do
48:    $c_n \leftarrow \begin{cases} 1, & \text{if } E_n \leq 0 \\ 0, & \text{otherwise} \end{cases}$ 
49: end for
50: Output:  $\mathbf{c}$ 

```

Algorithm 2 SIMD Processing for LDPC Decoding

```

1: Input: Received codewords  $Y$ , parity-check matrix  $H$ 
2: for each physical core  $p$  in parallel do
3:   Split codeword batch into  $P$  subsets  $\{Y_p\}$ 
4:   for each subset  $Y_p$  using SIMD width  $F$  do
5:     Load  $F$  codewords into vector registers
6:     while iterations  $<$  max_layered_iters do
7:       Parallel CN updates via HL-MS algorithm
8:       Variable node (VN) updates with SIMD
9:       Early termination by signs of layers
10:    end while
11:  end for
12: end for

```

a 2D array architecture. This integration enhances memory alignment, computational throughput, and SIMD utilization, overcoming challenges in efficient LDPC decoding.

- **Prefetching and Predicated Operations:** The design leverages AVX-256/512 prefetching and predicated operations (e.g., `_mm512_mask_blend`) to eliminate branching by applying conditional masks directly in vectorized operations. This approach minimizes control flow disruptions, enhancing decoding efficiency.
- **Reordering of CPM:** To optimize memory alignment and data locality, circulant permutation matrices are re-ordered. This preserves the first layer of the circulant permutation, preventing unwanted shifts and enhancing system efficiency.
- **1D VN Loading:** The algorithm maps the first dimension of the array to the SIMD vector register width ($F = 512/8 = 64$ bits for AVX-512). The second dimension size is $\lceil Z/F \rceil = 128/64 = 2$, where Z is the expansion factor. Memory alignment is achieved through zero-padding, ensuring that each Variable Node (VN) group becomes a fixed-size block $\lceil Z/F \rceil \times F$, even when $Z \bmod F \neq 0$. The overhead introduced by padding is negligible, as Z is typically large.

This SIMD-based approach significantly improves performance in high-throughput applications like DVB-S2 systems. By efficiently aligning and handling VN data, it reduces memory access latency and boosts throughput. Moreover, the integration of the 2D array architecture with SIMD processing optimizes computational resource utilization, ensuring better scalability and performance in LDPC decoding. This method represents a major advancement in decoding efficiency, making it particularly well-suited for modern, data-intensive communication systems.

Benchmarking

The system simulation ran for 600 seconds to estimate the throughput and latency of the LDPC decoder implementations. Using the proposed algorithm, the implementation achieved exceptional performance on a dual AMD EPYC-9754 system with 256 cores, an MZ73 motherboard, and 24 modules of

Algorithm 3 SIMD Processing for LDPC Decoding

Require: Total VNs (Z), SIMD width (F), Expansion factor ($G = \lceil Z/F \rceil$), Rotation offsets (R), VN memory layout

- 1: Initialize Registers $\leftarrow \emptyset$ $\triangleright$ Holds SIMD registers for VN data
- 2: Divide VN memory into $G \times F$ blocks, padded with zeros for alignment
- 3: Rotated_Blocks[G][F] $\leftarrow \emptyset$ $\triangleright$ Stores aligned data
- 4: **procedure** SIMDLLOADWITHROTATION(Z, F, G, R)
- 5: **for** $i = 0$ to $G - 1$ **do** $\triangleright$ Iterate over groups of VNs
- 6: **for** $j = 0$ to $F - 1$ **do** $\triangleright$ Iterate over SIMD lanes
- 7: Compute rotation offset $r \leftarrow R[i][j]$
- 8: Load SIMD: Block₁ $\leftarrow$ VN[$i \cdot F : (i+1) \cdot F - r$]
- 9: Load SIMD : Block₂ $\leftarrow$ VN
- 10: Apply masking operation (*blendv*):
Rotated_Block[i][j] $\leftarrow$ Mask(Block₁, Block₂, r)
- 11: **end for**
- 12: **end for**
- 13: **end procedure**
- 14: **procedure** PROCESSSIMDREGISTERS(Rotated_Blocks)
- 15: **for** $i = 0$ to $G - 1$ **do** $\triangleright$ Iterate over VN groups (rows)
- 16: **for** $j = 0$ to $F - 1$ **do** $\triangleright$ Iterate over SIMD lanes (columns)
- 17: Load rotated data into SIMD register:
Register[i][j] $\leftarrow$ Rotated_Blocks[i][j]
- 18: Perform CN $\rightarrow$ VN updates using SIMD instructions
- 19: Compute $\min_1, \min_2$
- 20: Accumulate updated VN results back into SIMD registers
- 21: **end for**
- 22: **end for**
- 23: **end procedure**
- 24: **procedure** WRITEBACK(Rotated_Blocks, Z, F, G)
- 25: **for** $i = 0$ to $G - 1$ **do** $\triangleright$ Iterate over VN groups
- 26: **for** $j = 0$ to $F - 1$ **do** $\triangleright$ Iterate over lanes
- 27: Write updated VN data back to memory: VN[$i \cdot F + j$] $\leftarrow$ Register[i][j]
- 28: Rotate and align data for the next iteration
- 29: **end for**
- 30: **end for**
- 31: **end procedure**

64GB SK Hynix DDR5-4800MHz RAM. The results, summarized in Table II, highlight the scalability and efficiency of the proposed approach in modern high-performance computing environments. The proposed decoder demonstrates substantial improvements in throughput for LDPC decoding, especially in high-throughput applications like DVB-S2 systems. By leveraging memory-efficient techniques and AVX-512 SIMD operations, our approach achieves a per-core throughput of 63.95 Mbps (2.81 watts), significantly outperforming (throughput per watt) previous implementations such as the 8.6 Mbps

(10.6 watts) per core on the EPYC-7351 system [7] and the 79.65 Mbps (7.5 watts) per core on Intel Xeon Gold 6148 processors [10]. These results highlight the advantages of the proposed decoder design, including enhanced parallelism, reduced memory overhead, and optimized energy consumption, making it highly suitable for modern multi-core processors and scalable LDPC decoding workloads.

IV. GPU DECODER WITH CN VALUE REUSE

TABLE II: DVB-S2 codes Throughput, 8 Iterations

Code Rate	Code Length	Inf. Throughput (Gbps)
$R = 1/2$	64800	25.49
$R = 3/4$	64800	35.7
$R = 2/9$	64800	12.13
$R = 37/45$	16200	59.96
$R = 11/15$	16200	48.92

The proposed GPU Decoder integrates an efficient forward-backward (FB) message-passing mechanism with a memory-optimized processing approach specifically tailored for GPU architectures. This method addresses the challenges of memory bottlenecks, parallelism, and data alignment that are commonly encountered in LDPC decoding. The key achievement of the memory-reduced approach lies in improving the efficiency of CN updates in LDPC decoding by effectively balancing computation and memory usage Alg. 4. The traditional two-minimum sum method for CN updates requires $4d_c$ comparisons. By splitting the CN update into forward, backward, and merging processes, the FB approach reduces the total number of comparisons to $3(d_c - 2)$. This reduction leads to computational savings that are proportional to the degree d_c , thus improving efficiency. In the FB approach, rather than storing separate forward (F_j) and backward (B_j) messages, memory space is shared between these messages during the process. This reduces the storage requirement for intermediate messages from $2(d_c - 2)$ to $d_c - 2$, optimizing memory usage and reducing overhead. By leveraging the GPU architecture, CN processing is assigned to thread blocks, and memory allocations are optimized through the use of shared memory and registers. This strategy minimizes memory bandwidth bottlenecks, ensuring high throughput and efficient use of the available computational resources. The proposed GPU Decoder with Memory-Optimized Forward-Backward Processing represents a significant advancement in LDPC decoding, Alg. 5. By integrating memory principles with GPU-specific optimizations, the proposed decoder achieves high throughput, low latency, and efficient memory utilization. This design offers a scalable and robust solution for modern error-correction systems, effectively bridging the gap between theoretical performance and practical implementation. The forward and backward message-passing processes are synchronized, ensuring minimal idle time for GPU cores and maximum utilization of available resources.

The hierarchical memory allocation minimizes data transfer costs and enhances memory throughput. The proposed GPU decoder with memory-optimized forward-backward processing

TABLE III: GPU peak performance and 802.11n QC-LDPC codes (1944, 972) decoders throughput

Reference	GPU	Peak Performance	Decoding parameters	Throughput (Gbps)
[16]	GTX TITAN X	11 TFLOPS FP32, 44 TOPS INT8	Flooding Min-Sum decoding, 10 iterations	0.913 Gbps
[17]	GTX TITAN X	11 TFLOPS FP32, 44 TOPS INT8	QC-LDPC with BCH, 10 iterations	1.66 Gbps
[18]	GTX 1080 Ti	8.873 TFLOPs FP32, Not support INT8	LNMS with early termination, 5 iterations	3.672 Gbps
[18]	GTX 1080 Ti	8.873 TFLOPs FP32, Not support INT8	Flooding with early termination, 5 iterations	4.7 Gbps
[18]	GTX 1080 Ti	8.873 TFLOPs FP32, Not support INT8	LNMS without early termination, 20 iterations	1153 Mbps
[18]	GTX 1080 Ti	8.873 TFLOPs FP32, Not support INT8	Flooding without early termination, 20 iterations	708 Mbs
This work	RTX 4090	82.58 TFLOPs FP32, 660.6 TOPS INT8	QLNMS decoding, 10 iterations	60.87 Gbps
This work	RTX 5090, 572.16	104.8 TFLOPs FP32, 3352 TOPS INT8	QLNMS decoding, 6 iterations	104.9 Gbps

Algorithm 4 Check Node Processing: FB NB vs 2-Min-Sum**Require:** Input messages $U_{p_i c}$, degree of check node d_c **Ensure:** Output messages $V_{c p_i}$ **Traditional Min-Sum Approach:**

- 1: Find the two smallest magnitudes m_1, m_2 among $|U_{p_i c}|$, $i \in \{0, \dots, d_c - 1\}$.
- 2: **for** $i = 0$ to $d_c - 1$ **do**
- 3: Compute $V_{c p_i} = m_1 + \text{scale} \cdot \prod_{j \neq i} \text{sgn}(U_{p_j c})$.
- 4: **end for**
- 5: **Total comparisons:** $4d_c$. Binary MS decoding.

Memory-Reduced Forward-Backward (FB) Approach:

- 6: **Backward Pass:** Compute intermediate backward messages B_i :
- 7: **for** $i = d_c - 2$ to 0 **do**
- 8: $B_i = \min(B_{i+1}, |U_{p_i c}|)$
- 9: **end for**
- 10: **Forward Pass and Merging:** Compute forward messages F_i and $V_{c p_i}$:
- 11: **for** $i = 1$ to $d_c - 1$ **do**
- 12: $F_i = \min(F_{i-1}, |U_{p_i c}|)$
- 13: $V_{c p_i} = \min(F_{i-1}, B_{i+1}) \cdot \prod_{j \neq i} \text{sgn}(U_{p_j c})$
- 14: **end for**
- 15: **Total comparisons:** $3(d_c - 2)$, [21], binary ver. [22].
- Memory Optimization:** Shared memory is reused for F_i and B_i , reducing storage from $2(d_c - 2)$ to $d_c - 2$, [23].

TABLE IV: 4090 GPU information bits (K) throughput under 6-Layered Iteration, FER = 10^{-5} in 6G Prototype

Name of Code	R	N	Z	Punc VN	K , Gbps
6G Prototype	$R = 4/5$	1920	128	0	60.87
802.11n Wi-Fi	$R = 1/2$	1944	81	0	66.77
5G BG1 Fam.3	$R = 2/3$	5376	384	2	24.41
5G BG1 Fam.3	$R = 1/3$	26112	384	2	7.45

demonstrates significant performance improvements across various LDPC code families, as shown in Table IV. Our implementation on an NVIDIA RTX 4090 GPU achieves throughput rates of up to 66.77 Gbps for Wi-Fi 802.11n codes and 60.87 Gbps for a 6G prototype under a 6-layered decoding schedule constructed, [11]–[15]. For longer 5G NR BG1 family codes, the decoder sustains impressive throughput levels of 24.41 Gbps and 7.45 Gbps for $R = 2/3$ and $R = 1/3$, respectively. These results establish the decoder's scalability and efficiency for both short and long block-length LDPC codes. In [16], a throughput of 913 Mbps for Wi-

Fi 802.11n (1944, 972) codes was reported using a GTX TITAN X GPU under flooding Min-Sum decoding with 10 iterations. The same hardware achieved 1660 Mbps when combined with BCH codes for concatenated decoding [17]. In contrast, our decoder achieves 66.77 Gbps for similar 802.11n codes, reflecting a speed-up of over 40 $\times$. In [18], a throughput of 4.7 Gbps was achieved for LDPC decoding on a GTX 1080 Ti GPU, which has a peak performance of 8.873 TFLOPs (FP32). This was based on 20 iterations with early termination. By comparison, our implementation achieves a throughput of 60.87 Gbps for 6 iterations on the RTX 4090, leveraging its peak performance of 82.58 TFLOPs (FP32), Tab. III. This represents scaled to peak performance a speed-up of approximately 13 $\times$. The NVIDIA RTX 4090 GPU's theoretical peak performance (82.58 TFLOPs FP32, 330.3 TOPS INT8) suggests a maximum achievable speed-up of 4 $\times$ over the GTX 1080 Ti based purely on hardware capabilities. However, the observed speed-up of over 13 $\times$ for 6-layered iterations underscores the efficiency of our implementation, which effectively bridges the gap between theoretical and practical performance. Even without CUDA, AI RISC-V optimization, the Nvidia 5090 achieved an throughput of 104.9 Gbps.

V. CONCLUSION

This work establishes new performance frontiers for QC-LDPC decoding on modern CPUs and GPUs through hardware-aware optimizations targeting SIMD parallelism, memory hierarchy utilization, and layered message-passing. The proposed architecture with AVX vectorization achieves 72.9 Gbps throughput for DVB-S2 codes (16200 bits) and 50.9 Gbps for 64800-bit codewords on dual AMD EPYC-9754 CPUs—a 5–12 $\times$ improvement over the AFF3CT framework with MIPP SIMD Wrapper. These results demonstrate that multi-core x86 architectures can deliver FPGA-class throughputs when combined with cache-aligned data structures and predicated SIMD operations eliminating branching penalties. On GPUs, our register-centric forward-backward check node update strategy achieves 66.77 Gbps for Wi-Fi 802.11n codes and 24.41 Gbps for 5G NR BG1 codes on RTX 4090—surpassing prior GPU implementations by 40–120 $\times$ [16], [18]. Using 802.11n QC-LDPC codes (1944, 972), an RTX 5090 GPU demonstrated a throughput of 104.9 Gbps. Three innovations enable this breakthrough. SIMD-optimized message rotation using AVX masked blend instructions to eliminate unaligned memory access penalties. Register-based CN processing reducing global memory transactions compared to

Algorithm 5 GPU Decoder with Memory-Optimized (Value Reuse) Check Node Forward-Backward (FB) Processing

Require: LDPC Code Parameters (Z , F , Rotation Offsets (R), Parity Check Matrix H), V2C ($U_{p,c}$), C2V ($V_{cp,i}$)

```

1: Initialization: Load  $H$  into shared memory from constant
   memory. Allocate GPU registers for intermediate back-
   ward ( $B_i$ ) and forward ( $F_i$ ) messages. Reserve shared
   memory for processing of ( $U_{p,c}$ ) and ( $V_{cp,i}$ ).
2: procedure 2DSYSTPROCESSING( $Z$ ,  $F$ ,  $R$ ,  $H$ ,  $U_{p,c}$ ,  $V_{cp,i}$ )
3:   for  $j = 1$  to  $d_c - 2$  do    ▷ Iterate over check node
   layers (degree  $d_c$ )
4:     Backward Pass:    ▷ Process blocks in reverse
   order
5:     for  $i = \lceil Z/F \rceil - 1$  to  $0$  do
6:       Compute  $B_i = \min(B_{i+1}, |U_{p,c}[i]|)$  using
   shared memory. Store  $B_i$  for merging with  $F_i$ .
7:     end for
8:     Forward Pass and Merging: ▷ Process blocks
   in forward order
9:     for  $i = 0$  to  $\lceil Z/F \rceil - 1$  do
10:      Compute  $F_i = \min(F_{i-1}, |U_{p,c}[i]|)$ 
11:      Merge  $F_i$  and  $B_i$  to compute  $V_{cp,i} =$ 
 $\min(F_{i-1}, B_{i+1}) \cdot \prod_{j \neq i} \text{sgn}(U_{p,j,c})$ .
12:    end for
13:  end for
14: end procedure
15: procedure OPTIMMEMACCESS( $Z$ ,  $F$ ,  $H$ ,  $U_{p,c}$ ,  $V_{cp,i}$ ,  $B_i$ )
16:   Load  $H$  into shared memory for efficient access. Store
 $U_{p,c}$  in shared memory blocks aligned with group size
( $F$ ).
17:   if  $Z > \text{SharedMemoryLimit}$  then
18:     Store partial  $V_{cp,i}$  messages into global memory for
   large LDPC codes.
19:   else
20:     Retain all  $V_{cp,i}$  messages in shared memory for
   small to medium LDPC codes.
21:   end if
22: end procedure
23: procedure EFFICIENTREGISTERSTORAGE( $B_i$ ,  $F_i$ )
24:   Allocate GPU registers for  $B_i$  and  $F_i$  for rapid access.
   Unroll loops for processing  $d_c$ -degree rows to minimize
   instruction overhead.
25: end procedure
26: procedure UPDATECNMESSAGES( $Z$ ,  $F$ ,  $U_{p,c}$ ,  $V_{cp,i}$ )
27:   for  $k = 0$  to  $F - 1$  do    ▷ Process  $F$ -wide lanes in
   parallel
28:     Load  $V_{cp,i}[k]$  from shared memory into registers.
   Perform parallel updates for all messages in blocks and
   write  $V_{cp,i}[k]$  back to shared/global memory.
29:   end for
30: end procedure

Final Decoding: If the signs of all layers  $> 0$ , com-
pute and output the decoded LDPC codeword. Otherwise,
return a decoding failure.

```

traditional two-minimum approaches. Decoupled parallelism exploiting both intra-codeword vectorization and inter-frame multicore, multiprocessor scaling. The results validate that software-defined QC-LDPC decoders can meet 6G throughput targets ($> 100\text{Gbps}$) using GPUs, make comparable to FPGA implementations, simulation while retaining full protocol flexibility. Future work will extend and improve this result to high efficient FPGA/ASIC terminal and base-station LDPC implementations.

REFERENCES

- [1] M. Z. Chowdhury, et al., "6G Wireless Communication Systems: Applications, Requirements, Technologies, Challenges, and Research Directions," OJ-COMS, vol. 1, pp. 957-975, 2020
- [2] P. Kamble and A. N. Shaikh, "6G Wireless Networks: Vision, Requirements, Applications and Challenges," ICAST, 2022, pp. 577-581.
- [3] H. Tataria, et al., "6G Wireless Systems: Vision, Requirements, Challenges, Insights, and Opportunities," in Proceedings of the IEEE, vol. 109, no. 7, pp. 1166-1199, 2021.
- [4] M. W. Akhtar, et al., 6G Use Cases, Requirements, and Metrics, Springer, 2020, pp. 1-10.
- [5] C. She, et al. Ultra-Reliable and Low-Latency Communications in 6G: Challenges, Solutions, and Future Directions, Springer, 2022, pp. 505-515.
- [6] C. X. Wang et al., "On the Road to 6G: Visions, Requirements, Key Technologies, and Testbeds," in IEEE Communications Surveys & Tutorials, vol. 25, no. 2, pp. 905-974
- [7] E. Grayver, "Scaling the Fast x86 DVB-S2 Decoder to 1 Gbps," IEEE Aerospace Conference, 2019, pp. 1-9
- [8] Cassagne A., et al. AFF3CT: A Fast Forward Error Correction Toolbox, SoftwareX, Volume 10, 2019, 100345
- [9] Cassagne A. et al. MIPP:a Portable C++ SIMD Wrapper and its use for Error Correction Coding in 5G Standard, WPMVP, 2018, No. 2, pp. 1-8
- [10] B. Le Gal, "Scalable High-Throughput and Low-Latency DVB-S2(x) LDPC Decoders on SIMD Devices," OJ-COMS, v. 5, 7216-7227, 2024
- [11] Svistunov G., et al. "Interleaved Cyclic Group Decomposition Length Adaptive MET QC-LDPC Codes," BlackSeaCom, 2019, pp. 1-3.
- [12] Svistunov G., Usatyuk V., "Protograph Sieving Method for Construction Moderate Length Multi-Edge Type QC-LDPC Codes," EWDTS, 2019, pp. 1-4.
- [13] V. Usatyuk and I. Vorobyev, "Construction of High Performance Block and Convolutional Multi-Edge Type QC-LDPC codes," TSP, 2019, pp. 158-163.
- [14] V. S. Usatyuk, "Low Error Floor QC-LDPC Codes Construction Using Modified Cole's Trapping Sets Enumerating," DSPA, 2023, pp. 1-6
- [15] V. S. Usatyuk et al., "Mixed Integer Linear Programming Method for Energy-Based Model Trapping Sets Enumerating," DSPA, 2024, pp. 1-6.
- [16] S. Keskin and T. Kocak, "GPU-Based Gigabit LDPC Decoder," in IEEE Communications Letters, vol. 21, no. 8, pp. 1703-1706, Aug. 2017
- [17] S. Keskin and T. Kocak, "GPU accelerated gigabit level BCH and LDPC concatenated coding system," HPEC, 2017, pp. 1-4
- [18] J. Yuan and J. Sha, "4.7-Gb/s LDPC Decoder on GPU," in IEEE Communications Letters, vol. 22, no. 3, pp. 478-481, 2018
- [19] T. J. Richardson and R. L. Urbanke, "Efficient encoding of low-density parity-check codes," IT, vol. 47, no. 2, pp. 638-656
- [20] A. Voicila, D. Declercq, F. Verdier, M. Fossorier and P. Urdard, "Low-complexity decoding for non-binary LDPC codes in high order fields," in IEEE Trans. on Comm., vol. 58, no. 5, pp. 1365-1375, 2010
- [21] Usatyuk V.S., Yegorov S.I. Construction of quasi-cyclic non-binary LDPC codes, based on joint optimization of distance properties and connection spectra of codes, Telecomm., 2016., Vol. 8., pp. 32-40.
- [22] Z. Wang and Z. Cui, "A Memory Efficient Partially Parallel Decoder Architecture for QC-LDPC Codes," Asilomar, pp. 729-733, 2005.
- [23] K. K. Gunnam, et. al. "A Parallel VLSI Architecture for Layered Decoding for Array LDPC Codes," VLSID, 2007, pp. 738-743
- [24] Richardson T., Urbanke R. Modern Coding Theory. / Cambridge University Press. - 2008, - 590 p.
- [25] Ryan W., Shu Lin Channel Codes: Classical and Modern / Cambridge University Press. - 2009, - 710 p.
- [26] Chen J., Tanner R. M., Jones C., Li Y. "Improved min-sum decoding algorithms for irregular LDPC codes," ISIT, 2005, pp. 449-453.